
VFF INTEGRATED MANAGEMENT SYSTEM

System Requirements Specification

Group No: C
Group Member IDs: S/21/444
S/21/145
S/21/029
S/21/301

Date: 27/02/2026
Revisions: Version 1.0

CSC3213 - Project in Computer Science I
Department of Statistics and Computer Science
Faculty of Science
University of Peradeniya

Version History

Version	Date	Modified By	Reviewed By	Modification Details
1.0				Initial version

Table of Contents

1. Introduction	5
1.1. Purpose	5
1.2. Scope.....	5
1.3. Data Flow Diagram.....	6
1.4. Definitions and Acronyms	7
1.5. References	7
2. General Description.....	8
2.1. System Functions	8
2.2. User Characteristics.....	8
3. General Constraints.....	10
3.1. Software Constraints	10
3.2. Hardware Constraints.....	10
4. Assumptions and Dependencies	11
5. Functional Requirements Master List.....	12
6. Detailed Functional Requirements.....	13
6.1. FR-REQ-5.1	Error! Bookmark not defined.
6.1.1. Description.....	15
6.1.2. System Input	15
6.1.3. System Processing.....	16
6.1.4. System Output	16
6.1.5. Other.....	16
6.1.6. Constraints	16
6.1.7. Data Handling	16
6.1.8. Error Handling.....	16
6.2. FR-REQ-5.2	Error! Bookmark not defined.
6.2.1. Description.....	Error! Bookmark not defined.
6.2.2. System Input	Error! Bookmark not defined.
6.2.3. System Processing.....	Error! Bookmark not defined.
6.2.4. System Output	Error! Bookmark not defined.
6.2.5. Other.....	Error! Bookmark not defined.
6.2.6. Constraints	Error! Bookmark not defined.
6.2.7. Data Handling	Error! Bookmark not defined.
6.2.8. Error Handling.....	Error! Bookmark not defined.
7. External Interface Requirement.....	25
7.1. Data Interfaces.....	25
7.2. User Interfaces.....	25

7.3. Other Interfaces.....	26
8. Non-Functional Requirements	27
8.1. System Performance.....	27
8.2. Information Security	27
8.3. Availability	27
8.4. Capacity.....	28
9. Functional Testing/Use-cases	29

1. Introduction

1.1. Purpose

This document is a detailed system requirements specification for the VFF Integrated Management System (IMS). The primary objective of this SRS is to completely define the technical, functional, and non-functional requirements necessary to develop and deploy the web-based application. This document will build on the Business Requirements Specification (BRS) and must have a corresponding High-Level Design (HLD) document. It serves as the definitive contractual baseline between the development team (Group C) and the stakeholders at Vets For Future (VFF). Any required changes to the Business Requirements Specification or the High-Level Design that arise in developing this System Requirements Specification (SRS) must be updated in those documents. Current and valid System Requirements Specification (SRS), High-Level Design (HLD) and Low-Level Design (LLD) are required as part of the operational support documentation set for any application.

1.2. Scope

This section describes the scope of the system. Define what is out of scope as well. The VFF IMS is a centralized, internal web application engineered to digitize and automate the core operational workflows of Vets For Future.

- **In Scope:** The system encompasses four tightly integrated vertical modules: Human Resources (Employee Profiles and Auto-Locking Attendance), Financial Management (Automated Payroll and Cash vs. Receivables Dashboards), Operations (Program-Centric Inventory Issuance and Returns), and System Administration (Role-Based Access Control and Audit Logging). The system will calculate specific organizational efficiency metrics, namely the "Cost-Per-Animal."
- **Out of Scope:** The system is strictly an internal administrative tool. Public-facing websites, external donor/volunteer registration portals, e-commerce integrations, and third-party API integrations (such as direct bank payroll routing) are explicitly out of scope for this release. Additionally, the manual migration of VFF's historical paper records into the new database is excluded from the development team's scope.

1.3. Data Flow Diagram

A dataflow diagram is included to demonstrate between what components the data is transferred.

Figure 1: Level 0 Context Data Flow Diagram illustrating external entity interactions with the VFF Integrated Management System.

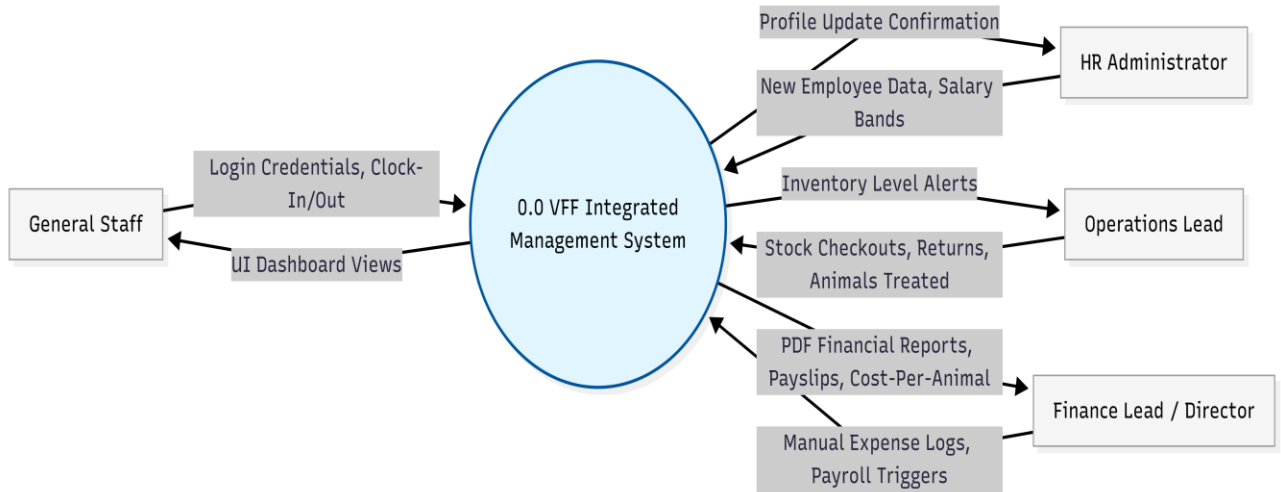
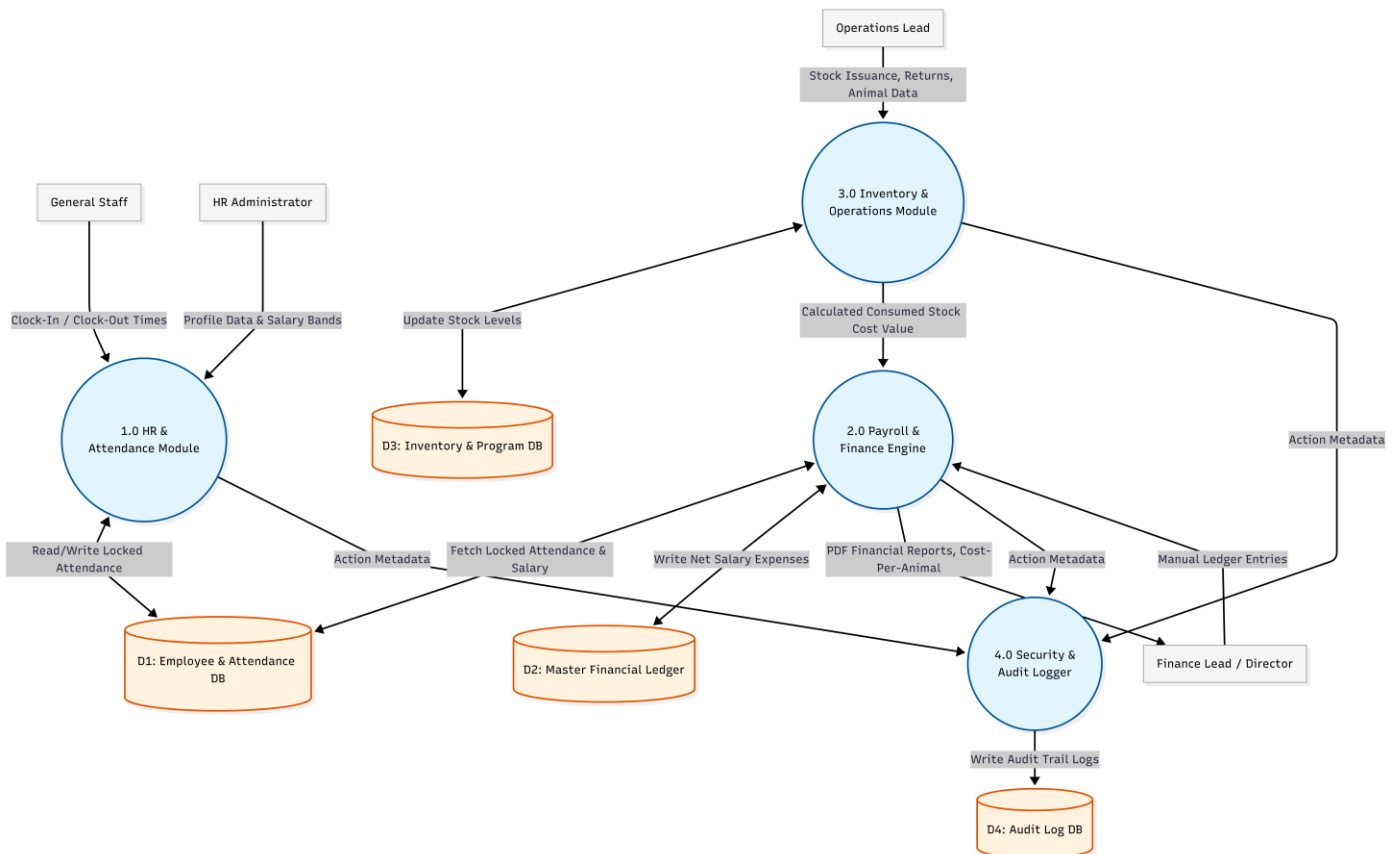


Figure 2: Level 1 System Breakdown Data Flow Diagram illustrating internal data routing between core modules and databases.



1.4. Definitions and Acronyms

This section describes any terms used.

- **API:** Application Programming Interface. A set of rules allowing the frontend UI to communicate with the backend database.
- **BRS:** Business Requirements Specification. The preceding document outlining the client's high-level business needs.
- **HLD / LLD:** High-Level Design / Low-Level Design. Architectural documents that will follow this SRS detailing the exact database schemas and software components.
- **IMS:** Integrated Management System. The comprehensive software solution being developed.
- **RBAC:** Role-Based Access Control. A security paradigm where system access is strictly restricted based on the user's assigned organizational role.
- **SRS:** System Requirements Specification. This current document.
- **VFF:** Vets For Future. The non-profit animal welfare organization acting as the primary client and end-user.
- **WBS:** Work Breakdown Structure. A project management methodology dividing the system into manageable vertical slices.

1.5. References

This section lists any references and includes pre-requisite documentation and any other supporting documentation such as diagrams.

- Department of Statistics and Computer Science, University of Peradeniya - *CSC3213 Project Guidelines*.
- Group C - *VFF IMS Project Proposal and Presentation Pitch* (Feb 2026).
- Group C - *VFF_RTM_Final.xlsx (Requirements Traceability Matrix)*.

2. General Description

2.1. System Functions

The Finance module of the VFF Integrated Management System (IMS) is the operational core that enforces financial discipline, eliminates ambiguity between available liquidity and expected income, and ensures rule-based payroll automation. This module is not a passive record-keeping system; it actively enforces financial logic.

The system shall provide the following financial functions:

- Maintain a real-time General Ledger that records all financial transactions (income, expense, payroll, inventory-linked costs, and program allocations).
- Distinguish clearly between:
 - Available Cash (actual liquid funds in bank/cash accounts), and
 - Outstanding Receivables (invoiced but unpaid amounts).
- Automatically update financial balances upon transaction approval.
- Link payroll expenses directly to the HR attendance engine using rule-based logic.
- Allocate all expenses to either:
 - Organizational overhead, or
 - Specific programs (for cost-per-animal and profit/loss computation).
- Generate monthly financial statements including:
 - Cash Position Summary
 - Accounts Receivable Aging Report
 - Payroll Summary
 - Program-Level Profit/Loss Report
- Prevent manual overrides of calculated payroll totals after finalization.
- Maintain a transaction-level audit trail for every financial modification.

Financial Logic Principles Enforced by the System:

1. No payroll is generated without locked attendance data.
2. No expense can exist without categorization.
3. Receivables do not increase available cash until payment confirmation is recorded.
4. Inventory consumption reduces stock and increases program expense simultaneously.
5. Once a payroll cycle is finalized, it becomes read-only.

This module transforms VFF from spreadsheet-based tracking into a rule-enforced financial system.

2.2. User Characteristics

Director

- Requires strategic-level financial visibility.
- Uses dashboards to monitor liquidity and program sustainability.
- Does not enter transactions but approves payroll cycles.
- Expects high-level summaries, graphs, and monthly statements.

Finance Officer

- Performs daily ledger entries.
- Reviews receivables and reconciles payments.
- Initiates payroll generation.
- Needs structured forms with validation logic and clear transaction flows.

HR Officer

- Maintains attendance data feeding into payroll engine.
- Must understand payroll rule enforcement but cannot override calculations.

System Administrator

- Configures financial rules (salary scales, incentive grading, tax deductions).
- Maintains user roles and permissions.

All financial users require a professional desktop-based web interface with structured forms, validation messages, dashboard analytics, and secure login authentication. The interface must prioritize clarity of cash flow representation and minimize cognitive load during transaction entry.

3. General Constraints

3.1. Software Constraints

This section lists the software constraints, including licensing, technical standards, and preferred technologies.

- **SC-01: Open-Source Licensing:** To minimize operational costs for VFF (a non-profit organization), the system must be developed utilizing exclusively open-source frameworks, libraries, and relational database management systems (e.g., MySQL or PostgreSQL). Proprietary, paid enterprise software licenses are strictly prohibited.
- **SC-02: Browser Compatibility:** The frontend User Interface (UI) is constrained to web-based delivery. It must be fully functional and visually responsive across modern, standard web browsers including Google Chrome (v90+), Mozilla Firefox (v88+), and Apple Safari (v14+).
- **SC-03: Compatibility:** The software will be hosted on cloud infrastructure such as AWS, Google Cloud, or Heroku, with MySQL or PostgreSQL for database management. The system should be able to scale based on usage, especially with increasing data volume and user count over time.
- **SC-04: Third-Party Dependencies:** The Monthly Financial Report feature (FR-10) is constrained by the capabilities of the selected open-source PDF generation microservice (e.g., Puppeteer or wkhtmltopdf). Document layouts must be designed within the rendering limits of these specific HTML-to-PDF engines.
- **SC-05: Web Standards:** The system will comply with modern web standards, including security protocols (e.g., HTTPS) for data transmission. It will not support outdated or legacy web browsers such as Internet Explorer or versions of Chrome/Firefox older than v90/v88, respectively.

3.2. Hardware Constraints

This section lists the hardware constraints, covering server and client-side physical requirements.

- **HC-01: Server Hosting:** The VFF IMS will be hosted on a cloud-based infrastructure (e.g., AWS, Google Cloud, or similar providers). This infrastructure must support scalable hosting to accommodate the increasing data volume, users, and financial transaction records.
- **HC-02: Client-Side Hardware Independence:** The system must not require VFF to purchase any specialized biometric hardware (e.g., fingerprint scanners or RFID badge readers) for the Daily Attendance Tracking (FR-03). All interactions must be executable via standard keyboard, mouse, or touch-screen inputs.
- **HC-03: Server Memory Constraints:** The cloud hosting environment selected for deployment must provision adequate RAM to support the memory-intensive PDF generation microservice. If deployed on a low-tier/free-tier cloud server (e.g., 512MB RAM), the system must queue PDF generation requests to prevent server out-of-memory (OOM) crashes.
- **HC-04: Mobile Access:** While the primary user experience is designed for desktop usage, mobile devices (smartphones/tablets) may be used for limited interactions. The system will be accessible via modern mobile web browsers, but certain advanced functionalities might be restricted compared to desktop use.

4. Assumptions and Dependencies

This section describes the critical assumptions and technical dependencies made in forming this system requirement. If any of these underlying conditions change, the scope, timeline, and functional delivery of the VFF Integrated Management System (IMS) may be impacted.

4.1. Assumptions

Assumptions are operational or business-level conditions that the development team (Group C) believes to be true for the successful deployment of the system.

- **A-01: End-User Literacy:** It is assumed that the administrative and general staff at Vets For Future (VFF) possess basic digital literacy and are capable of navigating standard web-based applications without requiring extensive, multi-week IT training.
- **A-02: Hardware Provisioning:** It is assumed that VFF will provide and maintain the necessary on-site physical hardware (e.g., standard desktop PCs, laptops, or shared tablets) required for staff to log their daily attendance and manage inventory.
- **A-03: Legacy Data Migration:** It is assumed that the manual migration of legacy organizational data (historical paper invoices, old employee records, past inventory logs) into the new digital database is the responsibility of the client, not the development team.
- **A-04: Language and Localization:** It is assumed that the system's User Interface (UI) and all generated PDF reports will be exclusively in English, and multi-language localization (e.g., Sinhala or Tamil translation toggles) is not required for the Minimum Viable Product (MVP).
- **A-05: Regulatory Compliance:** It is assumed that VFF management will handle any external legal or tax compliance regarding the financial data generated by the system; the IMS is strictly a data-recording tool, not an authorized accounting entity.

4.2. Dependencies

Dependencies are external technical factors, third-party systems, or infrastructural components that the VFF IMS fundamentally relies upon to execute its functional requirements.

- **D-01: Network Connectivity:** As a centralized web application, the system relies heavily on continuous, stable internet connectivity at the VFF operational sites. The system does not currently support an "offline-first" syncing architecture; database reads and writes require real-time server connections.
- **D-02: Cloud Hosting Infrastructure:** The deployment and uptime of the system depend on a reliable third-party cloud service provider (e.g., AWS, Google Cloud, Heroku) for web hosting and relational database management (e.g., MySQL/PostgreSQL).
- **D-03: Client-Side Software Support:** The UI relies on the end-user utilizing modern web browsers (e.g., Google Chrome v90+, Mozilla Firefox v88+, Safari v14+) with JavaScript fully enabled. Older browsers (such as Internet Explorer) are strictly unsupported.
- **D-04: Third-Party Microservices:** The system's Monthly Financial Report Generation (FR-10) depends on the continuous availability and compatibility of a backend PDF rendering library or microservice (e.g., Puppeteer, wkhtmltopdf, or TCPDF) to convert HTML/CSS ledgers into binary .pdf files.
- **D-05: Server Time Synchronization:** The Auto-Locking Attendance Tracking (FR-03) relies entirely on the host server's internal clock being accurately synchronized via Network Time Protocol (NTP) to Coordinated Universal Time (UTC) to prevent timestamp spoofing.

5. Functional Requirements Master List

This section lists the functional requirements in summary form. Each of these will be further defined in subsequent sub-sections (6.1, 6.2 etc.). Any **essential requirement** that contains at *least one of input, output, business logic, data handling, and error handling* can be considered as a functional requirement. **Try to limit the functional requirement to 20.**

Req. ID	Requirement Name	Requirement Description
FR-REQ-5.1	User Authentication & RBAC	The system shall enforce secure login and Role-Based Access Control.
FR-REQ-5.2	Employee Profile Management	The system shall allow administrators to manage staff records and salary bands.
FR-REQ-5.3	Daily Attendance Tracking & Auto-Locking	The system shall record daily attendance and lock the record upon submission.
FR-REQ-5.4	Automated Payroll Calculation	The system shall calculate monthly salaries based on locked attendance data.
FR-REQ-5.5	Real-time Cash vs. Receivables	The system shall display financial dashboards separating cash from pending invoices.
FR-REQ-5.6	Expense & Revenue Logging	The system shall allow manual entry of financial transactions into the ledger.
FR-REQ-5.7	Program-Centric Inventory Issuance	The system shall track inventory deducted specifically for assigned programs.
FR-REQ-5.8	Inventory Return & Stock Adjustment	The system shall process returned unused stock and adjust program costs.
FR-REQ-5.9	"Cost-Per-Animal" Calculation	The system shall calculate specific costs per animal using inventory and expenses.
FR-REQ-5.10	Monthly Financial Report Generation	The system shall generate professional, VFF-branded PDF financial reports.
FR-REQ-5.11	System Audit Logging	The system shall securely log all critical data modifications and deletions.

6. Detailed Functional Requirements

6.1. FR-REQ-5.1 User Authentication & Role-Based Access Control (RBAC)

6.1.1. Description

The system establishes a secure perimeter utilizing a token-based authentication architecture. It strictly enforces RBAC at both the frontend routing level (UI visibility) and backend middleware level (API endpoint protection).

6.1.2. System Input

1. email: VARCHAR(255), validated via regex for standard email formatting.
2. password: VARCHAR(255), minimum 8 characters, alphanumeric + special character requirement.

6.1.3. System Processing

- A. System receives credentials via an HTTP POST /api/auth/login request.
- B. Backend queries the users table for the email.
- C. Executes cryptographic comparison using Bcrypt (cost factor 10 or 12) against the stored hash.
- D. Upon match, generates a JSON Web Token (JWT). The JWT payload embeds the user_id and role_id (1=Admin, 2=Director, 3=HR Lead, 4=Staff).
- E. Middleware applies the RBAC matrix, blocking any API requests where the JWT's role_id lacks endpoint permissions.

6.1.4. System Output

- An HTTP 200 OK response returning the JWT (stored in sessionStorage or HttpOnly cookie), triggering a React/Angular state update to render the role-specific dashboard.

6.1.5. Other

6.1.6. Constraints

- Authentication tokens possess a strict Time-to-Live (TTL) of 30 minutes, after which a re-authentication challenge is triggered.

6.1.7. Data Handling

- Passwords are never logged in memory arrays or server logs. Input streams are sanitized to prevent SQL injection (e.g., utilizing parameterized queries or an ORM like Prisma/Sequelize).

6.1.8. Error Handling

- HTTP 401 Unauthorized: Triggered for incorrect passwords or emails, displaying a generic "Invalid Credentials" UI toast.
- HTTP 403 Forbidden: Triggered if a valid user attempts to access an endpoint outside their RBAC matrix.

6.2. FR-REQ-5.2 Employee Profile Management

6.2.1. Description

A complete CRUD (Create, Read, Update, Delete) module for HR personnel to manage the organizational structure, acting as the primary data dependency for the Payroll Engine.

6.2.2. System Input

1. first_name, last_name: VARCHAR(100).
2. nic: VARCHAR(20) (National Identity Card number, unique constraint).
3. base_salary: DECIMAL(10,2).
4. status: ENUM('Active', 'Inactive', 'Suspended').

6.2.3. System Processing

1. UI transmits a POST or PUT request to /api/employees.
2. Backend validates data types and executes a uniqueness check on the nic and email columns.
3. Executes insertion/update in the employee_profiles table.
4. "Deletions" execute a soft-delete logic (updating status to 'Inactive' rather than executing a SQL DELETE command) to preserve historical data integrity for past payroll cycles.

6.2.4. System Output

- HTTP 201 Created or 200 OK. UI dynamically re-renders the Employee Data Grid without requiring a full page reload.

6.2.5. Other

6.2.6. Constraints

- The base_salary data packet is stripped from the API response payload unless the requesting user holds the 'Director' or 'HR Lead' role.

6.2.7. Data Handling

- Salary inputs are strictly cast to floating-point decimals to prevent integer truncation. All text fields undergo HTML entity encoding to prevent Cross-Site Scripting (XSS).

6.2.8. Error Handling

- HTTP 409 Conflict is returned if an administrator attempts to register an NIC that already exists in the database.

6.3. FR-REQ-5.3 Daily Attendance Tracking & Auto-Locking

6.3.1. Description

An immutable timekeeping ledger. To guarantee data fidelity for the financial module, attendance records undergo an irreversible state transition upon submission.

6.3.2. System Input

1. employee_id: INT (Foreign Key).
2. target_date: DATE.
3. clock_in, clock_out: DATETIME (captured via system button click, not manual text entry).

6.3.3. System Processing

1. Employee clicks "Clock Out" via the UI, triggering a PATCH request.
2. Backend calculates worked_hours using the delta between clock_in and clock_out.
3. System executes a transaction updating the attendance table: UPDATE attendance SET clock_out = ?, worked_hours = ?, is_locked = 1 WHERE id = ?.
4. Any subsequent PUT/PATCH requests evaluating against a record where is_locked = 1 are programmatically terminated at the middleware level.

6.3.4. System Output

- Database commit confirming the locked state. The UI updates the row status indicator to a locked padlock icon.

6.3.5. Other

6.3.6. Constraints

- The system strictly rejects attendance submissions mapped to future calendar dates or dates preceding the employee's official join date.

6.3.7. Data Handling

- All time calculations utilize Coordinated Universal Time (UTC) at the database layer, converted to local Sri Lankan Time (IST/UTC+5:30) exclusively at the UI presentation layer.

6.3.8. Error Handling

- If an employee forgets to clock out before midnight, a cron job automatically flags the record as 'Incomplete' and locks it, requiring formal HR intervention.

6.4. FR-REQ-5.4 Automated Payroll Calculation

6.4.1. Description

The system shall automatically generate payroll based on attendance records, salary definitions, and rule-based incentive grading. Payroll calculations must be deterministic, reproducible, and protected from post-finalization modification.

6.4.2. System Input

1. Employee base salary
2. Attendance records (locked monthly)

-
3. Incentive grading parameters
 4. Deduction rules (e.g., unpaid leave penalties)
 5. Payroll period selection

6.4.3. System Processing

The payroll engine shall:

- A. Validate that attendance for the selected period is locked.
- B. Retrieve total payable working days.
- C. Compute gross salary using:
$$\text{Gross Salary} = (\text{Base Salary} / \text{Standard Working Days}) \times \text{Payable Days}$$
- D. Apply incentive multipliers if grading criteria are satisfied.
- E. Apply deductions based on defined rules.
- F. Compute net salary.
- G. Create a payroll ledger entry.
- H. Allocate payroll expense to either:
 - Administrative overhead, or
 - Specific program (if employee is program-assigned).

The system must enforce:

- No manual adjustment after finalization.
- Regeneration only allowed if payroll cycle is reset by authorized Director role.

6.4.4. System Output

- Individual Payslip (PDF format)
- Payroll Summary Report
- Updated General Ledger entry
- Updated Program Cost Allocation

6.4.5. Other

6.4.6. Constraints

- Payroll cannot be processed twice for the same period.
- Attendance modification invalidates payroll and requires recalculation.

6.4.7. Data Handling

- All salary values stored as decimal(precision,2).
- Unicode support for employee names.
- Payroll records permanently stored for auditing.

6.4.8. Error Handling

- Attempt to process unlocked attendance → System blocks operation.
- Missing salary configuration → System displays validation error.
- Duplicate payroll period → Operation rejected.

6.5. FR-REQ-5.5 Real-time Cash vs. Receivables

6.5.1. Description

The system shall provide a real-time dashboard that clearly distinguishes liquid cash from expected income (receivables). This feature directly addresses liquidity risk management

6.5.2. System Input

1. Confirmed payment transactions
2. Issued invoices
3. Expense entries
4. Payroll ledger entries

6.5.3. System Processing

The system shall:

1. Aggregate all confirmed income transactions → Update Available Cash.
2. Aggregate all unpaid invoices → Update Outstanding Receivables.
3. Subtract total expenses from available cash.
4. Display:

$\text{Available Cash} = \text{Confirmed Income} - \text{Confirmed Expenses}$ $\text{Receivables} = \text{Total Invoiced} - \text{Payments Received}$
5. Provide aging breakdown for receivables:
 - 0–30 days
 - 31–60 days
 - 61+ days

The system must prevent receivables from inflating cash position.

6.5.4. System Output

Dashboard must display:

- Available Cash (highlighted)
- Total Receivables
- Receivable Aging Chart
- Monthly Cash Flow Trend Graph
- Liquidity Warning Indicator (if cash below threshold)

6.5.5. Other

6.5.6. Constraints

- Only confirmed transactions affect cash.
- Directors have read-only access.

6.5.7. Data Handling

- All financial records timestamped.
- Aggregations computed dynamically.

6.5.8. Error Handling

- Data inconsistency detected → System logs anomaly.
- Missing transaction references → Block entry until resolved.

6.6. FR-REQ-5.6 Expense & Revenue Logging

6.6.1. Description

The system shall track Accounts Payable and Accounts Receivable as two independent financial streams, ensuring that obligations (payables) and expected income (receivables) are recorded, monitored, and settled without mixing them into the available cash balance until payment occurs.

6.6.2. System Input

1. Payable entry details (vendor/supplier, invoice/reference number, amount, due date, category, program link optional)
2. Receivable entry details (payer/donor/client, invoice/reference number, amount, expected date, program link optional)
3. Payment confirmation details (payment date, payment method, receipt/reference, amount paid)
4. Partial settlement details (remaining balance, settlement notes)

6.6.3. System Processing

The system shall:

1. Create a Payable record when an expense is committed but not yet paid, storing due date and outstanding balance.
2. Create a Receivable record when income is invoiced/expected but not yet received, storing expected date and outstanding balance.
3. Maintain separate ledgers/views for:
 - Payables: total outstanding, due soon, overdue
 - Receivables: total outstanding, expected soon, overdue
4. Support partial settlements:
 - If a payment amount is less than outstanding, reduce balance and keep the record open.
 - If payment equals outstanding, mark the record as settled/closed.
5. On payment confirmation:
 - For Receivables: convert the settled amount into confirmed income and update Available Cash.
 - For Payables: convert the settled amount into confirmed expense and reduce Available Cash.
6. Enforce consistency rules:
 - A receivable cannot directly increase Available Cash until payment confirmation is recorded.
 - A payable does not reduce Available Cash until payment confirmation is recorded (but remains visible as obligation).
7. Provide aging categorization for both:
 - 0–30 days, 31–60 days, 61+ days (or overdue based on due/expected dates).
8. Log every create/update/settlement action in the audit log with before/after balances.

6.6.4. System Output

- Accounts Payable list view (Open, Due Soon, Overdue, Settled)
- Accounts Receivable list view (Open, Expected Soon, Overdue, Settled)
- Payable aging summary + overdue indicator
- Receivable aging summary + overdue indicator

- Settlement receipt record (linked to payable/receivable)
- Updated financial dashboard values (Available Cash, Outstanding Receivables, Payables Exposure)

6.6.5. Other

6.6.6. Constraints

- Payables and receivables must remain separate categories and cannot be merged into a single “pending” list.
- Only authorized roles can confirm payments/settlements.
- A record cannot be marked “Settled” unless the settlement amount fully clears the outstanding balance.
- Duplicate invoice/reference numbers for the same counterparty must be blocked unless explicitly allowed by admin rule.

6.6.7. Data Handling

- Monetary values stored as decimal(precision,2).
- Every payable/receivable must include: reference number, counterparty, amount, date fields, status, and audit metadata.
- Status values must be controlled (e.g., Open, Partially Settled, Settled, Overdue).
- All lists and reports must support filtering by program, counterparty, status, and date range.

6.6.8. Error Handling

- Missing mandatory fields (reference number, amount, due/expected date) → Validation error and block submission.
- Payment confirmation exceeding outstanding balance → Reject and prompt correction.
- Attempt to confirm settlement without permission → Access denied and logged.
- Inconsistent state detected (negative outstanding balance, invalid status transitions) → Block operation, log anomaly, require admin review.

6.7. FR-REQ-5.7 Centric Inventory Issuance

6.7.1. Description

An operational module bridging physical logistics with financial tracking, ensuring drawn stock is mathematically converted into program expenditure.

6.7.2. System Input

1. item_id: INT.
2. issue_quantity: INT.
3. target_program_id: INT.

6.7.3. System Processing

1. Backend initiates an ACID-compliant SQL Transaction (BEGIN).
2. Evaluates `SELECT current_stock FROM inventory WHERE item_id = ? WITH (UPDLOCK)`.
3. If `current_stock >= issue_quantity`, executes `UPDATE inventory SET current_stock = current_stock - ?`.

-
4. Calculates `financial_value = issue_quantity * unit_cost`.
 5. Executes `INSERT INTO program_expenses (program_id, amount, expense_type)` using the calculated value.
 6. System executes `COMMIT`.

6.7.4. System Output

- Simultaneous decrementation of physical stock counts and incrementation of operational program liabilities.

6.7.5. Other

6.7.6. Constraints

- Stock quantities are strictly maintained as non-negative integers.

6.7.7. Data Handling

- Implementing pessimistic row-level locking during the database read/write cycle ensures that if two users attempt to draw the last remaining item simultaneously, race conditions are prevented.

6.7.8. Error Handling

- If `current_stock < issue_quantity`, the system executes `ROLLBACK`, terminating the transaction and returning an HTTP 422 Unprocessable Entity ("Insufficient Stock Available").

6.8. FR-REQ-5.8 Inventory Return & Stock Adjustment

6.8.1. Description

A reconciliation mechanism executed at the conclusion of field operations, restoring unused physical assets and recursively correcting program financial data.

6.8.2. System Input

1. `item_id`: INT.
2. `return_quantity`: INT.
3. `source_program_id`: INT.

6.8.3. System Processing

1. System queries the `inventory_transactions` table to sum the total quantity of `item_id` historically issued to `source_program_id`.
2. Algorithm verifies `return_quantity <= total_issued_quantity`.
3. Initiates an SQL Transaction to increment `inventory.current_stock` by the return amount.
4. Calculates the reimbursement value (`return_quantity * unit_cost`) and writes a contra-entry (negative expense) to the `program_expenses` table.

6.8.4. System Output

- Restored master stock equilibrium and dynamically reduced programmatic costs.

6.8.5. Other

6.8.6. Constraints

- Returns cannot be processed for consumable assets that are flagged in the database as 'Single-Use' or 'Perishable' beyond their expiration date.

6.8.7. Data Handling

- Audit trails for returns must explicitly reference the original Issuance Transaction ID to maintain a clear chain of custody.

6.8.8. Error Handling

- Returns an HTTP 400 Bad Request ("Return quantity exceeds original issuance") if the validation logic fails.

6.9. FR-REQ-5.9 "Cost-Per-Animal" Calculation

6.9.1. Description

The pinnacle analytical algorithm of the operations suite, aggregating disparate data streams (inventory, direct cash, payroll mapping) to output a unified efficiency metric.

6.9.2. System Input

1. program_id: INT
2. total_animals_treated: INT

6.9.3. System Processing

1. System executes a multi-table JOIN query to aggregate: Sum of direct program cash expenses + Sum of net unreturned inventory value + (Optional) Prorated staff hourly costs mapped to the program.
2. Mathematical Engine evaluates: $\text{Total_Aggregated_Cost} / \text{total_animals_treated}$.

6.9.4. System Output

- A single, highly precise floating-point metric injected into the Operations UI Dashboard and formatted for PDF reporting.

6.9.5. Other

6.9.6. Constraints

- Algorithm execution is disabled until the specific Program is formally marked with the status 'Completed'.

6.9.7. Data Handling

- To prevent architectural strain, this complex calculation is executed dynamically upon request rather than constantly recalculating via background workers.

6.9.8. Error Handling

- Implements an explicit if (total_animals_treated === 0) catch block to intercept mathematical DivisionByZero exceptions, throwing a UI alert requesting accurate animal data.

6.10. Monthly Financial Report Generation (PDF)

6.10.1. Description

A document compilation engine that transforms raw JSON ledger data into a flattened, heavily formatted, and legally archivable PDF binary.

6.10.2. System Input

1. report_month: INT
2. report_year: INT

6.10.3. System Processing

1. Controller aggregates required ledger, payroll, and inventory data arrays.
2. Injects data arrays into a predefined server-side HTML/CSS template mechanism (e.g., Handlebars.js or EJS).
3. Pipes the rendered HTML into a headless browser instance (e.g., Puppeteer) or PDF library (e.g., wkhtmltopdf).
4. The library processes the DOM, executes print CSS media queries, and generates a binary PDF buffer.

6.10.4. System Output

- An HTTP response with Content-Type: application/pdf and Content-Disposition: attachment, prompting a native browser file download.

6.10.5. Other

6.10.6. Constraints

- The PDF generation microservice is isolated to prevent heavy CPU loads from lagging the primary web server thread.

6.10.7. Data Handling

- The HTML template utilizes embedded base64 strings for organizational logos rather than external URL links to guarantee the PDF renders correctly even if offline.

6.10.8. Error Handling

- If the PDF buffer exceeds the maximum allocated memory or times out (>10 seconds), the server returns HTTP 504 Gateway Timeout, advising the user to reduce the date range.

6.11. System Audit Logging

6.11.1. Description

An overarching, invisible forensic architecture that shadows all core database modifications, guaranteeing strict compliance and data accountability.

6.11.2. System Input

- Automated background triggers capturing:
 - actor_user_id
 - http_method
 - target_table
 - row_id
 - old_payload_json
 - new_payload_json
 - ip_address

6.11.3. System Processing

1. Implemented via the application's ORM lifecycle hooks or native SQL triggers.
2. Before any INSERT, UPDATE, or DELETE commits to a protected table (e.g., financial_transactions, employee_profiles), the middleware serializes the before-and-after states into JSON strings.
3. Executes a secondary, synchronous INSERT INTO audit_logs.

6.11.4. System Output

- A continuous, append-only security ledger.

6.11.5. Other

6.11.6. Constraints

- The audit_logs table has database-level DENY UPDATE and DENY DELETE permissions applied universally, ensuring even compromised admin accounts cannot erase their tracks.

6.11.7. Data Handling

- Large JSON payloads are compressed before insertion into the audit_logs table to mitigate rapid database bloat.

6.11.8. Error Handling

- If the connection to the audit_logs table fails, the system executes a hard fail-safe: the primary user transaction is rolled back entirely to prevent any undocumented database changes.

7. External Interface Requirement

7.1. Data Interfaces

[EI-REQ-7.1.1] General Ledger API

The system shall expose internal service endpoints for:

- Payroll Ledger Creation
- Transaction Posting
- Receivable Registration
- Payment Confirmation
- Program Cost Allocation

These APIs shall:

- Require authentication token validation.
- Accept JSON payloads.
- Return structured response with status codes.
- Enforce transaction atomicity (no partial updates).

[EI-REQ-7.1.2] Payroll Engine Interface

Internal API for retrieving:

- Locked attendance records
- Salary configuration
- Incentive parameters

[EI-REQ-7.1.3] Reporting Interface

Service for generating:

- Payslip PDFs
- Monthly financial summaries
- Program cost reports

7.2. User Interfaces

[EI-REQ-7.2.1] Finance Dashboard UI

The financial dashboard shall include:

- Top Section:
 - Available Cash (Large visual indicator)
 - Outstanding Receivables
- Middle Section:
 - Monthly Cash Flow Graph
 - Receivables Aging Bar Chart
- Bottom Section:
 - Recent Transactions Table
 - Payroll Summary Snapshot

Color-coded indicators shall visually separate liquid funds from pending income.

[EI-REQ-7.2.2] Payroll Processing UI

Features:

- Period Selection Dropdown
- Preview Calculation Button
- Payroll Summary Table

-
- Finalize Payroll Button (Role Restricted)
 - Download Payslips

[EI-REQ-7.2.3] Ledger Entry UI

Features:

- Transaction Type Selector
- Expense Category Dropdown
- Program Link Option
- Validation Warnings
- Submit & Confirm Button

All forms shall provide:

- Inline validation
- Error highlighting
- Confirmation dialogs before finalization.

7.3. Other Interfaces

- Secure Database Connection (SSL-enabled)
- PDF Generation Service
- Internal Authentication Service

All financial operations must occur over encrypted communication channels.

8. Non-Functional Requirements

8.1. System Performance

This section describes the system performance requirements. Example response times are suggested though different systems requirements may apply.

Req. Id	Description	Response Time
NF-PERF-1.1	Time experienced by the user for regular actions such as navigating between fields, selecting menu items, or typing data.	≤ 1 second
NF-PERF-1.2	Time experienced when opening a new module screen, saving attendance records, or submitting standard data forms.	≤ 3 seconds
NF-PERF-1.3	Time experienced during significant processing such as payroll generation, cost-per-animal calculation, or financial dashboard refresh.	≤ 5 seconds
NF-PERF-1.4	Time experienced for report generation processes such as monthly financial summaries or bulk PDF payslip creation.	≤ 10 seconds for standard monthly dataset

8.2. Information Security

This section describes the information security requirements.

Req. Id	Description
NF-SEC-1.1	User credentials must be encrypted during transmission using secure communication protocols. Passwords must be stored using secure hashing mechanisms.
NF-SEC-1.2	Sensitive financial and employee data shall only be stored in secured database environments with restricted access controls. Development or testing environments must not contain real production data.
NF-SEC-1.3	The system shall enforce Role-Based Access Control to restrict access to financial dashboards, payroll processing, and administrative configuration features.
NF-SEC-1.4	All critical actions including payroll processing, attendance locking, financial record updates, and inventory issuance must be logged with timestamp and user identification for audit purposes.

8.3. Availability

This section describes the availability requirements in terms of days and permissible planned and un-planned un-availability.

Req. Id	Description
NF-AVA-1.1	The system shall maintain at least 99.5 percent (3.6 hours/month) availability during official working hours.
NF-AVA-1.2	Planned maintenance activities shall be communicated in advance and scheduled outside normal working hours where possible.

NF-AVA-1.3	In the event of unexpected system failure, recovery procedures must restore core functionality within 2 hours
NF-AVA-1.4	Database backup operations shall be performed daily to prevent data loss

8.4. Capacity

Req. Id	Description
NF-CAP-1.1	The system must be able to manage the following data volumes in year ONE. In Year One, the system must support up to 100 employees, 50 active programs, and 10,000 financial transaction records.
NF-CAP-1.2	The system must be able to manage the following data volumes in year TWO. In Year Two, the system must scale to support up to 200 employees, 100 programs, and 25,000 transaction records
NF-CAP-1.3	The system must be able to manage the following data volumes in year THREE. In Year Three, the system must scale to support up to 300 employees, 150 programs, and 50,000 transaction records without performance degradation
NF-CAP-2.1	The system must be able to manage the following concurrent user operation volumes in year ONE. In Year One, the system must support at least 20 concurrent users
NF-CAP-2.2	The system must be able to manage the following concurrent user operation volumes in year TWO. In Year Two, the system must support at least 35 concurrent users
NF-CAP-2.3	The system must be able to manage the following concurrent user operation volumes in year THREE. In Year Three, the system must support at least 50 concurrent users
NF-CAP-3.1	The database design shall allow horizontal scaling or migration to higher-capacity servers without redesigning the system architecture.

9. Functional Testing/Use-cases

Use case ID	Functional Requirement ID	Description	Input(s)	Expected Output(s)
FT-9.1	FR-REQ-5.1	Use Case Name: User Authentication & RBAC Actors: Director, Admin User Description: The system authenticates users and grants access based on assigned roles using Role-Based Access Control. Preconditions: User account exists; user role assigned. Postconditions: User is logged in and redirected to role-specific dashboard. Main Flow: 1. User enters credentials. 2. System validates credentials. 3. System identifies user role. 4. Appropriate dashboard is loaded. Alternative Flow: User requests password reset. Exception Flow: Invalid credentials → access denied and error message displayed.	Username, password	Successful login with role-based dashboard access
FT-9.2	FR-REQ-5.2, FR-REQ-5.3	Use Case Name: Employee Profile & Attendance Management Actors: Admin User Description: Admin manages employee records and records daily attendance, which becomes locked after submission. Preconditions: Employee profile exists; attendance date is open.	Employee details, Attendance data	Updated employee profile; Locked attendance record

		Postconditions: Attendance record stored and locked; audit log created. Main Flow: 1. Admin updates employee details if needed. 2. Admin records attendance. 3. Admin submits attendance. 4. System validates and locks record. Alternative Flow: Employee added before attendance submission. Exception Flow: Attempt to modify locked attendance → system blocks action.		
FT-9.3	FR-REQ-5.4	Use Case Name: Automated Payroll Processing Actors: Finance Admin, Director Description: System calculates monthly salaries using locked attendance records and predefined grading rules. Preconditions: Attendance locked; payroll configuration exists. Postconditions: Payroll summary generated; PDF payslips available. Main Flow: 1. Finance selects payroll period. 2. System retrieves attendance. 3. Salary rules applied. 4. Payroll summary generated. 5. Payslips produced. Alternative Flow: Director reviews payroll before confirmation. Exception Flow: Missing attendance data → payroll halted with validation message.	Locked attendance data, Salary rules	Payroll summary and downloadable payslips

FT-9.4	FR-REQ-5.5, FR-REQ-5.6	Use Case Name: Financial Dashboard & Ledger Entry Actors: Director, Finance Admin Description: System records financial transactions and displays a dashboard separating available cash from receivables. Preconditions: Financial records exist. Postconditions: Dashboard updated reflecting accurate financial position. Main Flow: 1. User logs transaction. 2. System validates and stores entry. 3. Dashboard updates. Alternative Flow: Bulk transaction entry. Exception Flow: Invalid transaction amount → validation error displayed.	Transaction data	Updated financial dashboard
FT-9.5	FR-REQ-5.7, FR-REQ-5.8	Use Case Name: Program-Centric Inventory Lifecycle Actors: Operations Admin Description: Inventory is issued to specific programs and adjusted upon return. Preconditions: Inventory available; program exists. Postconditions: Stock levels updated; program cost updated.	Program ID, Stock quantity	Updated inventory and program cost

		Main Flow: 1. Admin selects program. 2. Issues stock. 3. System deducts central inventory. 4. Returns processed if applicable. Alternative Flow: Partial return processed. Exception Flow: Insufficient stock → issuance denied.		
FT-9.6	FR-REQ-5.9	Use Case Name: Cost-Per-Animal Calculation Actors: Director, Operations Admin Description: System calculates cost per animal using direct expenses and inventory value used. Preconditions: Program data and expenses recorded. Postconditions: Cost-per-animal displayed. Main Flow: 1. User selects program. 2. System retrieves expense and stock data. 3. Calculation performed. Alternative Flow: Inventory adjustments applied before calculation. Exception Flow: Animals treated = 0 → calculation blocked.	Program expense data and inventory usage	Calculated cost per animal displayed on dashboard
FT-9.7	FR-REQ-5.10	Use Case Name: Monthly Financial Report Generation Actors: Director, Finance Admin	Month selection	Downloadable PDF report

		Description: System generates branded monthly PDF financial reports. Preconditions: Monthly data available. Postconditions: PDF report generated and stored. Main Flow: 1. Select month. 2. System compiles data. 3. PDF generated. Alternative Flow: Export to spreadsheet. Exception Flow: No data → user notified.		
FT-9.8	FR-REQ-5.11	Use Case Name: System Audit Logging Actors: System (automatic), Director Description: All critical system actions are logged for traceability. Preconditions: Sensitive action performed. Postconditions: Audit entry stored with timestamp and user ID. Main Flow: 1. Critical action performed. 2. System logs action automatically. Alternative Flow: Logs filtered by user/date. Exception Flow: Logging failure triggers system alert.	Critical system action	Audit log entry recorded

